

Product Roadmap

Phase 1: Foundation (Completed)

1. ☒ Core Asset Data Model — Asset objects as first-class entities with key-based identification M
2. ☒ SQLite Data Storage — Simple storage for asset data and computed analytics S
3. ☒ Yahoo Finance Data Downloader — Historical market data ingestion S
4. ☒ DAG Computation Framework — petgraph-based analytics dependency graph L
5. ☒ Push-Mode Analytics Engine — Incremental computation with dependency propagation XL
6. ☒ Basic Analytics Library — Moving averages, returns, volatility M
7. ☒ High-Speed Data Replay System — Historical replay at configurable speed M
8. ☒ Pull-Mode Analytics Engine — Time-series generation on-demand L
9. ☒ REST API Server with WebSocket/SSE — Real-time push updates M
10. ☒ React UI Dashboard — Real-time visualization and replay controls L

Phase 2: Core Event Engine (Current Focus)

11. ☐ Code Refactoring & Cleanup — Consolidate DAG node types, improve error handling, optimize push/pull integration M
12. ☐ Shared Product/Asset Domain Model — Define 14 products + 10 assets as shared Rust types between Ledger and Index frameworks L
13. ☐ Ledger Abstraction Layer — Common trait for in-memory and persistent implementations (persistence impl deferred) M
14. ☐ Ledger Core (In-Memory) — Double-entry bookkeeping, move generation, wallet/position model using abstraction layer L
15. ☐ Trigger Type System — Implement 8 trigger types (expiry, coupon, fixing, barrier, continuous barrier, American exercise, dividends, stock splits) L
16. ☐ Smart Contract Adapter (Thin Slice) — Stateless adapter with two-call pattern for continuous barrier products (Knock-Out Warrant, Mini Certificate) - leverages existing event framework logic XL
17. ☐ Move Rules Engine — Multi-dimensional context matching, booking patterns (Direct, WashBook, Split) L
18. ☐ Portfolio Tagging System — Position tagging, portfolio-as-filter queries M
19. ☐ Event Framework Stub — Simulate trigger registration, callback dispatch, two-call orchestration (continuous barrier already implemented) L
20. ☐ Quant Lib Stub — Mock product-level calculations for testing M

Phase 3: Full Product Coverage

21. ☐ Time-Based Products — Reverse Convertible (expiry, coupon, fixing) M

- 22. ☐ Discrete Barrier Products — Barrier Reverse Convertible, Bonus Certificate, Discount Certificate L
- 23. ☐ Periodic Fixing Products — Factor Certificate, Asian-style variants M
- 24. ☐ American Exercise Products — Vanilla Options, American-style warrants M
- 25. ☐ Complete Asset Coverage — All 10 hedging asset types M
- 26. ☐ Full Move Rules — TaxWithholding, Composite, Custom booking patterns M

Phase 4: Index Integration

- 27. ☐ Index as Ledger — Model index constituents as ledger positions L
- 28. ☐ Index Rebalancing Events — Trigger-driven constituent changes M
- 29. ☐ Index NAV Calculation — Real-time index level computation M
- 30. ☐ Decomposition/Flattening — Swap-on-index position to underlying exposures L
- 31. ☐ Index Research Mode — High-speed simulation for backtesting index rules L
- 32. ☐ Provisional Calculations — Pre-fixing index calculations marked as provisional, propagate through DAG but do not persist in windowing/recursive functions L

Phase 5: Production Hardening (OUT OF SCOPE - Separate Spec)

The following items are documented for completeness but are OUT OF SCOPE for this library. They will be addressed in a separate production hardening specification.

- 33. ☐ Ledger Persistence (Postgres) — Durable storage implementation behind abstraction layer L
- 34. ☐ WAL-Based Recovery — Write-ahead logging for crash recovery M
- 35. ☐ Idempotency & Audit Trail — Idempotent moves, full audit history M
- 36. ☐ Structured Logging — tracing-based observability S

Phase 6: Integration & Scaling (OUT OF SCOPE - Separate Spec)

The following items are documented for completeness but are OUT OF SCOPE for this library. They will be addressed in separate specifications.

- 37. ☐ Production REST API Layer — Production-grade API for external access M
- 38. ☐ Real-Time Event Ingestion — Live market data and corporate action feeds L
- 39. ☐ Unified Risk Aggregation — Flattened exposure views across ledgers L
- 40. ☐ Performance Optimization — Parallel execution, caching, memory optimization L
- 41. ☐ Distributed Architecture — Horizontal scaling, node coordination XL

Phase 7: Python & External Integration

- 42. ☐ Python PyO3 Bindings — Direct Rust-to-Python access for research workflows L
- 43. ☐ Polars Dataframe Integration — Analytics export as Polars dataframes S
- 44. ☐ Python REST Client — Python client library with Polars integration S

Notes - Phase 1 (Foundation) validated push/pull analytics with real-time UI - Phase 2 (Core Event Engine) establishes the unified architecture with Ledger as central abstraction - Ledger Abstraction Layer comes BEFORE implementation to ensure in-memory and future persistent versions share the same interface - **Thin slice uses continuous barrier products** (Knock-Out Warrant, Mini Certificate) because this trigger logic already exists in the event framework - Phase 3 completes structured product coverage (time-based, discrete barriers, fixings, exercise) - Phase 4 extends the same paradigm to index operations - **Provisional Calculations:** Before official fixings are available, indices can be calculated with results marked as provisional. These values propagate through the DAG for downstream consumers but are NOT persisted in stateful operations (windowing functions, recursive calculations, moving averages). When the official fixing arrives, it replaces the provisional value and IS persisted. - **Phase 5 & 6 are OUT OF SCOPE** — documented for architecture visibility but will be separate specs - Phase 7 adds Python ecosystem integration for research workflows - Each phase builds on previous, maintaining the “same engine, different modes” principle - This library focuses on orchestration of quantitative analytics and business logic, NOT production infrastructure